

IoT-Based Bus Stop System for Public Transport Overcrowding Prediction Using Machine Learning

CHAPTER 2

LITERATURE REVIEW AND REQUIREMENT ANALYSIS

2.1 INTRODUCTION

Chapter 1 introduced the background of this project, identified the recurring overcrowding risk faced by commuters at busy public bus stops such as Dindoshi Bus Depot (serving BEST Route 326), and presented the proposed solution: an IoT-based bus stop system for overcrowding prediction using machine learning, combining a VL53L0X Time-of-Flight passenger counter, a DHT22 sensor, and live OpenWeatherMap weather data on an ESP32, feeding a Random Forest model that outputs a transparent risk score with LED and Telegram alerts. This chapter focuses on the analysis and planning required before development of the proposed system. It includes a review of existing research studies, a comparative analysis of available passenger-counting and transit tools, identification of the research gap, requirement gathering methods, stakeholder analysis, functional and non-functional requirements, a feasibility study, software and hardware requirements, the project schedule, and the development methodology adopted. A detailed analysis of these aspects helps in understanding the technical requirements, selecting suitable technologies, identifying limitations of existing solutions, and ensuring that the proposed system is practical, reliable, and capable of meeting the safety needs of the community before implementation begins.

2.2 LITERATURE REVIEW

Eight peer-reviewed studies were examined in terms of the problem addressed, methodology, result, and limitation or relevance to the proposed bus-stop crowding system.

Paper 1

Title: Data-driven Bus Crowding Prediction Models Using Context-specific Features (2020)

Authors: Tahereh Arabghalizi and Alexandros Labrinidis

This study addressed prediction of how full an approaching bus would be. The authors used operational records and contextual features including time, route direction, weather, and recent passenger loads to train regression and classification models. Context-specific Random Forest models improved crowding prediction over baseline approaches. The method supports the project's feature selection, but depends on extensive transit-agency data instead of direct sensing at one stop.

Paper 2

Title: iABACUS: A Wi-Fi-Based Automatic Bus Passenger Counting System (2020)

Authors: Michele Nitti, Francesca Pinna, Lucia Pintor, Virginia Pilloni, and Benedetto Barabino

iABACUS investigated passenger counting without ticket taps or manual observation. It captured Wi-Fi probe activity and filtered detected devices to estimate passengers. Testing reported 100% device-detection accuracy in static conditions and approximately 94% in dynamic conditions. Although inexpensive, the method depends on device ownership, Wi-Fi behaviour, and identifiers. The proposed project therefore uses anonymous short-range Time-of-Flight crossings.

Paper 3

Title: To Travel or Not to Travel: 'Weather' Is the Question. Modelling the Effect of Local Weather Conditions on Bus Ridership (2018)

Authors: Sui Tao, Jonathan Corcoran, Francisco Rowe, and Mark Hickman

This research combined bus-ridership records with local weather variables to model changes across places and times. The results showed that rainfall and temperature significantly affect ridership, with effects varying by time and location. This supports including rain, temperature, and hour in the feature vector. However, the work studied wider travel behaviour rather than real-time crowd risk at one shelter.

Paper 4

Title: An IoT Transport Architecture for Passenger Counting: A Real Implementation (2021)

Authors: Janine Kniess, Julio Cezar Rutke, and William Alberto Cruz Castaneda

This work implemented an IoT architecture that used embedded computer vision and networked processing to count boarding and alighting passengers. It demonstrated practical acquisition and transmission of passenger information through a connected system. However, camera counting introduces cost, lighting, occlusion, and privacy concerns. The proposed prototype instead uses a low-cost VL53L0X sensor without recording faces or images.

Paper 5

Title: IoT-Enabled Real-Time Monitoring of Urban Garbage Levels Using Time-of-Flight Sensing Technology (2025)

Authors: Luis Miguel Pires, Joao Figueiredo, Ricardo Martins, and Jose Martins

The researchers connected a VL53L8CX Time-of-Flight sensor to an ESP32-S3 to measure waste-bin fill levels and transmit readings through an IoT backend. Testing demonstrated accurate, low-power, real-time distance monitoring. Although applied to waste management, the study provides a technical precedent for combining ESP32 hardware and ToF sensing in a low-cost municipal monitoring node.

Paper 6

Title: Empirical Study on the Accuracy and Precision of Automatic Passenger Counting in European Bus Services (2019)

Authors: Alessandro Olivo, Giulio Maternini, and Benedetto Barabino

This study compared infrared boarding and alighting counts with manual observations in European bus services to evaluate accuracy and precision. The equipment was generally effective, but errors increased during difficult passenger flows, including closely spaced or simultaneous crossings. The findings show why counting must be validated under crowding and support confirmation logic, although the proposed stop-level sensor is simpler than commercial APC equipment.

Paper 7

Title: Development and Evaluation of Frameworks for Real-Time Bus Passenger Occupancy Prediction (2023)

Authors: Jonathan Wood, Zhengyao Yu, and Vikash V. Gayah

This study used linear regression and Random Forest with real-time operations, APC readings, and weather to predict occupancy at future stops. Origin-destination-pair models were more reliable downstream, while both model families produced close estimates. The work supports combining passenger and weather information, but depends on vehicle-level counters and historical agency records instead of direct measurement at one bus stop.

Paper 8

Title: Personalized Predictive Public Transport Crowding Information with Automated Data Sources (2020)

Authors: Erik Jenelius

This paper combined historical counts, real-time vehicle locations, APC data, regression models, and probabilistic seat allocation to estimate seating probability and standing time. A Stockholm bus-route case study showed that real-time counts improved prediction and could inform travellers early enough to be useful. The method relies on a data-rich network and onboard information rather than local sensing and public-facing shelter warnings.

Synthesis of Literature

The studies agree that automated passenger information becomes more useful when combined with temporal, operational, or environmental context. They differ in sensing inputs - Wi-Fi, cameras, infrared APC, operational records, and ToF - and in prediction methods such as regression, Random Forest, and probabilistic modelling. Data-rich systems provide detailed predictions, while low-cost sensing reduces infrastructure needs but requires calibration.

What Is Missing

Most reviewed systems predict occupancy inside vehicles or across networks using agency datasets, cameras, Wi-Fi, GPS, or installed APC equipment. Limited research combines anonymous sensing at one bus stop with temperature, live weather, time, ML risk classification, local indication, remote alerts, and logging. The proposed project fills this gap through

VL53L0X and DHT22 sensing, OpenWeatherMap, Random Forest, Blynk, LEDs, Telegram, and CSV logging in one low-cost prototype.

2.3 COMPARATIVE ANALYSIS OF EXISTING SYSTEMS

Five representative existing systems were selected because they cover commonly available commuter applications, physical bus-stop displays, low-cost experimental counters, and industrial passenger-counting products. They were compared using cost, standalone operation, stop-level focus, and local alert capability.

Table 2.1: Comparative Analysis of Existing Bus-Stop Crowding Solutions

System	Cost	Standalone?	Stop-Level Focus	Local Alerts
Chalo / Ridlr (transit apps)	Free	No; app-based	No; GPS vehicle tracking	App only
Google Maps Transit	Free	No; cloud-based	No; historical or cellular estimates	App only
Traditional Digital Bus Stop Displays	High	Partial; display only	Arrival time only	None
DIY ESP32 / Blynk Crowd Counters	Low	No; Wi-Fi or cloud needed	Partial; single sensor, no context	LED, buzzer, or app
Industrial APC (camera or IR fleet systems)	Very high	No; vendor system	Vehicle-level, not shelter-local	Central software
Proposed IoT-Based Bus Stop Crowding Prediction System	Low	Partial; local sensing, Wi-Fi for remote features	Yes; count, weather, time, and ML risk	LEDs, Blynk, and Telegram

The comparison shows that existing commuter applications and displays mainly provide arrival or vehicle information, while industrial APC systems are expensive and operate inside vehicles. Low-cost DIY counters usually provide only raw counts. None of the compared alternatives combines anonymous stop-level counting, environmental context, machine-learning risk classification, on-site indication, controlled alerts, operator correction, and local history in one affordable system.

2.4 RESEARCH GAP IDENTIFICATION

The literature review and comparative analysis reveal the following gaps:

- **Lack of stop-level multimodal prediction:** Most research operates at vehicle, route, or network level. Simpler embedded systems count people without combining passenger count, temperature, rainfall, and time into one stop-level risk score.
- **Vehicle-centric and cloud-only products:** Transit applications and industrial APC products primarily track vehicles or onboard passengers. They do not physically sense the waiting crowd or provide a public-facing indication at the shelter.

- **Limited low-cost alerting and operational control:** DIY counters rarely combine defined risk thresholds, local and remote alerts, departure correction, alert cooldown, and persistent records for later review.

The proposed system fills these gaps by combining a VL53L0X ToF sensor, DHT22, OpenWeatherMap information, and a RandomForestRegressor with an ESP32 and Python backend. Blynk presents passenger and risk information; LEDs provide local NORMAL or CRITICAL indication; Telegram provides controlled CRITICAL alerts; V7 applies the Bus Left (-30) correction; V8 shows time since the last crossing; and every processing cycle is appended to bus_stop_log.csv.

2.5 REQUIREMENT GATHERING METHODOLOGY

Observation, interviews, a structured commuter survey, and literature review were used to gather requirements for the proposed system. Combining these methods allowed both community needs and technical constraints to influence the final requirements.

Observation Method

Passenger movement and waiting conditions were observed at Dindoshi Bus Depot, particularly the BEST Route 326 boarding edge, during the morning peak from 8:00-10:00 AM and evening peak from 5:00-7:00 PM. Manual passenger-count tallies and written notes recorded crowd build-up, bus departures, weather exposure, and the limitations of visual judgement.

Interview Method

Informal one-to-one interviews were conducted with daily commuters and three depot staff members. Responses were recorded in written notes. Participants discussed peak-hour crowding, rain and heat, the time taken to notice dangerous build-up, useful warning methods, and how the count should be corrected after a bus departs.

Interview Questions Asked

1. What are the main difficulties you face when the bus stop becomes overcrowded during peak hours or bad weather?
2. How do you currently know whether the stop is crowded before reaching it?
3. How useful would an on-site LED indication and a remote dashboard or alert be?
4. After a bus leaves, how should the passenger count be corrected while retaining people who remain waiting?
5. Which information is most important: passenger count, weather, temperature, risk status, alerts, history, or operator control?

Survey Method

A structured survey was conducted using Google Forms with exactly 25 regular commuters at Dindoshi Bus Depot and the Route 326 boarding edge. The results showed that 84% preferred an on-site display instead of relying only on a mobile application, 92% wanted an alert within 30 seconds of critical crowding, and 76% expected the displayed count to decrease after a bus

departed rather than reset completely to zero. The most common concern was the absence of warning before overcrowding became severe.

Literature and Technical Review

Research papers, technical articles, platform documentation, and component datasheets were reviewed to evaluate passenger-counting technologies, prediction methods, weather integration, IoT communication, and alert delivery. The findings supported selection of the ESP32, VL53L0X, DHT22, Blynk, OpenWeatherMap, Telegram, Random Forest, two-reading crossing confirmation, five-second backend cycle, and the three risk states.

2.6 STAKEHOLDER ANALYSIS

This project involves several groups of people, each with a different relationship to the system and different expectations from it, as summarized below.

Stakeholder	Role	Need
Daily Commuters (Route 326 / Dindoshi)	Direct users	Simple, glanceable awareness of crowding (on-site LED indicators) without needing an app or technical knowledge
Transit / Depot Staff	Direct users	Clear on-site and remote risk indication to quickly judge severity and decide whether to intervene
Depot Authority / Transit Administrators	Decision-makers	Low-cost, low-maintenance solution that provides proactive alerts (Telegram) without requiring constant dashboard-watching
Field Maintenance Staff / Technicians	Direct users (hardware)	Low-cost, modular, easy-to-replace hardware (ESP32/I2C sensors) without specialized industrial training
Academic Evaluators (Project Guide / CEP Coordinator)	Evaluators / indirect stakeholders	A well-documented, technically sound system demonstrating correct application of IoT and machine learning concepts
Project Students (TYIT-A)	System developers	Build, test, and document the ESP32 + brain.py + Blynk + Telegram + CSV system end-to-end
Future Commuters / Wider Community	Long-term beneficiaries	Continued protection from undetected stop overcrowding after installation

Among these, daily commuters and transit staff are the most critical stakeholders, since they interact with the system daily. Commuters' preference for simple, glanceable awareness without installing an app directly shaped the decision to provide on-site LED indicators (with an audible buzzer present but firmware-disabled for public-safety reasons) alongside a colour-coded Blynk dashboard, rather than a phone-app-only solution. Transit staff's need for a remote risk signal without actively watching the dashboard shaped the Telegram alert escalation when risk exceeds the CRITICAL threshold of 75%, and their need to correct the passenger count after a bus departs — without erasing commuters who did not board — shaped the V7 "Bus Left (-30)" button supported by the V8 time-since-last-crossing evidence. The depot authority's need for a low-cost, subscription-free solution further reinforced the decision to use free-tier APIs (OpenWeatherMap, Blynk) and CSV-based logging instead of a paid cloud platform.

2.7 FUNCTIONAL REQUIREMENTS

Functional requirements define the specific behaviours, actions, and data-processing capabilities the system must perform.

- **FR-1:** The ESP32 shall detect a passenger crossing when two consecutive VL53L0X readings are < 400 mm and increment the passenger count.
- **FR-2:** The ESP32 shall read temperature/humidity from the DHT22 on GPIO4 and publish temperature to Blynk V4.
- **FR-3:** The ESP32 shall publish passenger count to Blynk V1 and minutes since last crossing to Blynk V8.
- **FR-4:** brain.py shall fetch OpenWeatherMap weather, map the condition (Rain/Drizzle/Thunderstorm/Shower) to Rain/Clear, and write the weather display to Blynk V2.
- **FR-5:** brain.py shall compute risk using the locked formula and RandomForestRegressor (100 trees, random_state=42), writing the score to Blynk V3.
- **FR-6:** The system shall classify risk as NORMAL (0–40), WARNING (41–75), or CRITICAL (>75), noting that exactly 75.0 is WARNING.
- **FR-7:** On CRITICAL, the red LED (GPIO25) shall activate and a Telegram alert shall be sent with a 60-second cooldown; green LED (GPIO26) indicates NORMAL conditions as implemented.
- **FR-8:** Blynk V7 push button "Bus Left (–30)" shall set passengerCount = $\max(0, \text{passengerCount} - 30)$.
- **FR-9:** brain.py shall append timestamp, passengers, temp, weather, risk, and status to bus_stop_log.csv every cycle.
- **FR-10:** The buzzer shall remain firmware-disabled despite being physically wired via an NPN transistor; alerts go to Telegram instead.

The above functional requirements define the core operations of the proposed system so that sensing, scoring, alerting, operator correction, and logging remain consistent with the implemented system.

2.8 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes, performance benchmarks, and design constraints of the system.

- **NFR-1 (Performance):** Sensor publish and brain cycles shall update Blynk gauges frequently enough for a live demo (count climb, risk rise, V7 response visible within normal demo timing).
- **NFR-2 (Public-safety UX):** No firmware audible alarm at the stop; CRITICAL escalation is visual (red LED) plus Telegram to authority.
- **NFR-3 (Usability):** Blynk layout shall place location label, V7, V8, gauges (V1 max 45; V3 max 98), weather/temp, status colour, and chart in a supervisor-readable arrangement.
- **NFR-4 (Integrity / transparency):** Risk caps (passenger component ≤ 75 ; overall ≤ 98) and thresholds shall be viva-defensible without claiming absolute certainty.
- **NFR-5 (Availability):** Prototype node and brain.py shall support continuous demo operation while powered and networked.
- **NFR-6 (Compatibility):** Stack uses ESP32 Arduino libraries, Blynk Cloud, Python 3, scikit-learn, and OpenWeatherMap HTTPS access.
- **NFR-7 (Scalability — future):** Architecture should allow additional stops later; cloud database remains future work beyond file logging.
- **NFR-8 (Maintainability):** Modular wiring on the 3.3 V rail (SDA21/SCL22, DHT GPIO4, LED 25/26 via 220 Ω) supports student maintenance and viva explanation.
- **NFR-9 (Cost-effectiveness):** Components and free APIs (OpenWeatherMap free tier with API key; Blynk free tier for prototype pins) keep deployment within academic budget.

Non-functional requirements ensure the prototype is not only technically capable but also community-safe, affordable, and honest — critical for CEP trust.

2.9 FEASIBILITY STUDY

A feasibility study determines whether the proposed overcrowding-prediction system can be successfully developed and demonstrated for Dindoshi / Route 326 within the academic CEP constraints.

Technical Feasibility

The system is technically feasible because it relies on well-documented hardware and software: ESP32 Wi-Fi, VL53L0X I2C ranging, DHT22, Blynk virtual pins, OpenWeatherMap JSON weather, scikit-learn RandomForestRegressor, and Telegram Bot API. Development tools (Arduino IDE, Python) are readily available, making integration achievable for TYIT students.

Economic Feasibility

The project is economically feasible using off-the-shelf ESP32-class components rather than industrial APC. OpenWeatherMap's free tier requires only a free API key; Blynk Cloud free-tier pins V1–V8 suffice for the prototype; CSV logging needs no paid database. This keeps cost within an academic CEP budget.

Operational Feasibility

Operators need minimal training: watch passenger and risk gauges, check V8 before pressing V7 after a bus leaves, and receive Telegram alerts when risk exceeds 75%. Commuters are not asked to install an app. The firmware-disabled buzzer avoids public panic. Therefore operational adoption for a supervised demo is realistic.

Schedule Feasibility

The work fits the Semester V CEP timeline through phases: requirement analysis, hardware assembly, firmware, brain.py + ML formula, Blynk dashboard, Telegram testing, CSV verification, documentation, and demo rehearsal. Parallelism between wiring and Python development shortens the critical path, with buffer for sensor tuning and network debugging.

2.10 SOFTWARE REQUIREMENTS

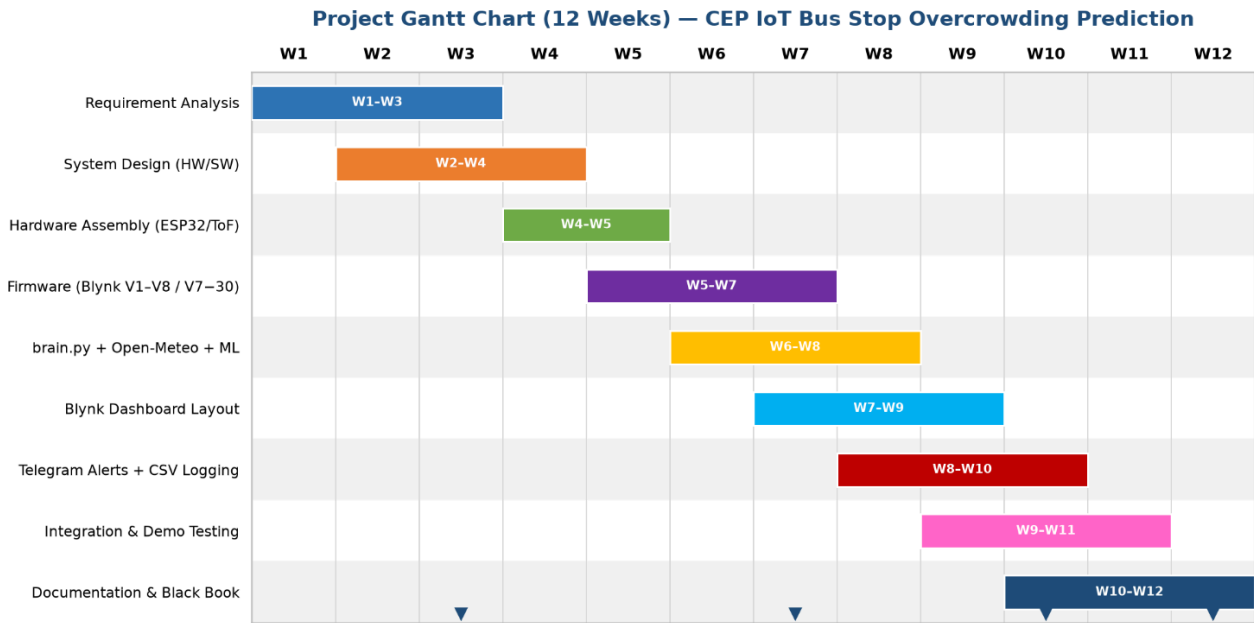
Software / Framework	Version	Purpose
Arduino IDE	2.x / ESP32 core	Writing and flashing C++ firmware to the ESP32 microcontroller
Blynk Cloud	Virtual pins V1–V8	Dashboard, button V7, gauges, chart
Python	3.10+	Data processing and running brain.py orchestration loop
scikit-learn	Latest (RandomForestRegressor)	Training and running the Random Forest model (100 trees, random_state=42)
OpenWeather Map API	v2.5	Fetching live, localized meteorological data (temperature, rain) over Wi-Fi; condition mapping (Rain/Drizzle/Thunderstorm/Shower → is_raining)
Telegram Bot API	Current	Sending real-time, push-based crowd risk alerts (>75%, 60s cooldown) to the transit authority
CSV logging	bus_stop_log.csv	File-based persistent database
Windows 11 / Linux	Current	Primary operating system for development and ML training

2.11 HARDWARE REQUIREMENTS

Hardware	Specification	Purpose
ESP32 DoIT DevKit V1	Wi-Fi enabled, 3.3V rail for peripherals	Central edge-computing unit for reading sensors and connecting to Blynk
VL53L0X ToF Sensor	Laser, I2C (SDA GPIO21 / SCL GPIO22)	Passenger crossing count (<400 mm × 2 consecutive readings)
DHT22 Sensor	Digital (GPIO4)	Monitors localized microclimate (temperature/humidity); heat bonus if $\geq 33^{\circ}\text{C}$
Red LED	GPIO25 via 220 Ω	CRITICAL indication
Green LED	GPIO26 via 220 Ω	NORMAL indication
Buzzer + NPN transistor	Wired path	Present but firmware-disabled
Misc power/wiring	Breadboard / USB power	Prototype assembly
Development Laptop	Core i5 / 8GB RAM min	Writing code, training the Random Forest model, running brain.py, monitoring serial output

2.12 PROJECT SCHEDULE

The project is structured across a 12-week development cycle. Phase 1 covers requirement analysis and hardware procurement; Phase 2 covers hardware assembly, breadboard prototyping, and sensor calibration; Phase 3 covers OpenWeatherMap API integration and ML model training; Phase 4 covers system integration, where hardware and the AI model are linked and debugged; and Phase 5 is reserved for final testing and documentation, which runs partly in parallel with Phase 4 so that testing and reporting are not left until the very end.



2.13 DEVELOPMENT METHODOLOGY

This project uses an Iterative Prototyping methodology: a basic, functional version of the system is built quickly, tested, and then refined through successive iterations.

This approach suits IoT and machine learning projects particularly well, because hardware sensors often behave less predictably in the field than in theory. Building a quick prototype lets the team test the VL53L0X sensor's actual field-of-view and calibrate it before writing the more complex AI software. Machine learning also benefits from this approach, since it requires ongoing tuning — an iterative cycle allows the team to train the Random Forest model, test it against real ESP32 sensor data, and retrain it to improve accuracy over time. Rejected ideas — automatic timer reset and count decay — were tested conceptually and discarded because they can wipe a real crowd mid-rush. This maps directly onto the schedule in Section 2.12, where hardware prototyping (Phase 2) is completed and tested before the heavier software engineering and AI integration work (Phases 3 and 4) is finalized.